

ComicCrafter: A Zero-Cost Multi-Agent AI System for Automatic Comic Generation from Text

Ahmed Bin Asim¹, Absir Ahmed Khan¹, and Fahd Ahmad Farooq¹

National University of Computer and Emerging Sciences,
Islamabad, Pakistan
{22i-0949, 22i-0915, 22i-1131}

Abstract. The process of creating comics based on text automatically is a difficult problem since it is necessary to be accurate in the story and maintain the visuals. LLMs and diffusion image generators are great individually, but it is not yet that old how to make them combined into a complete comic without exceeding constrained resources. We created ComicCrafter, a free, multi-agent, AI platform that transforms your text into a sequence of comic strips, which appear to be unified, with just free software. It operates on a LangGraph workflow to orchestrate multiple agents, one which divided the story, another which edited prompts, and also which planned the layout. And a RAG module that has ChromaDB to keep the characters looking like they should between panels. We also tie into free LLMs such as Groq Llama 3.3 and Google Gemini, free image backends such as GGML, HuggingFace and Replicate, meaning that we can remain on the free side without compromising quality. We experimented on ComicCrafter and found that it is consistent in its characters - DINO similarity 0.72 ± 0.08 - and in the story flow, CLIP-T score 0.68 ± 0.05 , even with the use of free-tier accounts. We discovered that LangGraph approach is 23-percent better than a straightline pipeline in terms of character consistency and 18-percent in terms of reliability. It demonstrates that advanced multimodal systems could be provided by intelligent design and resource juggling providing students and independent creators with a very powerful, free creative tool.

Keywords: Comic Generation · Multi-Agent Systems · Retrieval-Augmented Generation · LangGraph · Character Consistency · Story Visualization

1 Introduction

The convergence of the generative AI and creative storytelling has presented previously unavailable possibilities of automated content creation. Creation of comic books is among the most challenging multimodal processes as it must know the plotline and character development and have the visual elements aligned to each other within the panels and obey the rules of artistic composition [12]. In contrast to creating a single image, creating a comic means that the appearance

of the characters that take part in the comic must be consistently the same over the long term, that the scenes have to transition smoothly, and a mix of text (dialogue, captions, sound effects) and artwork must be properly mixed [6].

The recent developments in the fields of Large Language Models (LLMs) [4,?] and diffusion-based image generators [1,?] promise a lot. Nevertheless, producing a coherent comic out of them is a relatively untapped area, particularly when you are only left with free compute or open APIs to play with. The current approaches to visualising stories typically require GPUs, fine-tuning or special hardware, which may be difficult to afford by researchers, students, and hobbyists with limited budgets.

The current paper presents ComicCrafter, a free open-source multi-agent AI solution to the following question: How can we create coherent comics using a tight budget using free-tier diffusion model APIs and LLMs?

(3) Hybrid Backend Architecture: ComicCrafter communicates with a variety of free-tier image backends (local GGML using stable-diffusion.cpp, HuggingFace Inference API, Replicate API) and uses each one automatically. Such a multi-provider design makes this system continuously operate in the event that a single service reaches a rate limit, and also allows the user to select a backend that matches their hardware.

(4) Zero-Cost Optimization Strategies: We use premature-efficiency tricks, cache characters, and single-shot generation to reduce the number of API calls and maintain quality. These tricks allow the system to continue operating with heavy free-tier quotas.

(5) Evaluation Framework: The character consistency with DINO similarity and LPIPS distance is evaluated, the narrative flow with CLIP-T alignment and the comparison between various LLM providers and image backends is performed. Our findings indicate DINO similarity of 0.72 ± 0.08 and CLIP-T of 0.68 ± 0.05 which is comparable to methods that utilize pay-walled resources. Success is enhanced by LangGraph orchestration compared to direct coordination by 23%. We also like Groq Llama 3.3 70B that strikes the balance between speed (2.3s average response) and quality in decomposing stories, and local GGML that removes rate constraints but introduces latency.

The remainder of the paper is structured as follows: Section 2 reviews relevant literature on the multi-agent systems, story visualization and character-consistent generation. Section 3 describes the system architecture which consists of the LangGraph workflow, RAG integration and image generation pipeline. Section 4 gives the experimental setup, evaluations metrics and results. Section 5 will be a discussion of findings, limitations, and future work. Section 6 concludes the paper.

2 Related Work

2.1 Multi-Agent LLM Systems

The emergence of large language models has created an interest in multi-agent collaboration systems. Guo et al. [4] review the domain of LLM-based multi-

agent systems, and note such issues as agent communication protocols, task allocation, and conflict management. Tran et al. [9] consider the mechanisms of collaboration in multi-agent systems and suggest the hierarchical coordination patterns in which we operate in our story-to-comic pipeline. Han et al. [5] state that there are gaps in the use of multi-agent systems based on the LLM, such as the necessity to have stateful workflows with conditional branching and we address this gap by LangGraph.

Our work is based on those concepts by delegating roles with agents specialized in complementary activities (story analysis, prompt optimization, layout planning), which is more efficient and has better output quality than monolithic LLM solutions.

2.2 Story Visualization and Comic Generation

Diffusion models have much improved story visualization. Zhou et al. [12] developed StoryDiffusion that consists of self-attention that remains consistent through extended sequences so that the characters do not change their appearance. Their mechanism requires narrowing down on particular character datasets and GPUs, thus not working with everyone. The models presented by He et al. [6] are an open-domain system, though the researchers use proprietary versions of DALL-E, which contradicts the zero-cost rule.

Conversely, ComicCrafter adheres to workflows of the API complete attempts based on free-tier providers. It achieves this with both consistency by using RAG-powered prompt augmentation rather than fine-tuning, which is more available and yet competitive.

2.3 Character-Consistent Image Generation

Remaining character consistency in series of images is an enormous problem. The first publication is a segmentation-based reference conditioning and attention injection, done by Avrahami et al. [1] to maintain identity called The Chosen One. CharacterFactory proposed by Wang et al. [11] draws on the use of GANs on special consistency losses to sample characters.

Neither of these methods is as consistent but requires either reference images and a manual segmentation [1] or peons of training on character datasets [11]. We have a practical alternative in our RAG-based approach: there is no additional training needed, and our approach to inject consistency information into the prompts by querying a vector store and pulling out descriptions of characters. That loses some accuracy to the methods that are reference-conditioned, but puts the whole appearance in a zero-cost budget.

2.4 Retrieval-Augmented Generation (RAG)

Lewis et al. [7] were the first to apply the knowledge-intensive NLP to RAG systems via external documents and model weights. The modern RAG frameworks

have been overviewed by Gao et al. [3], emphasizing the retrieval and fusion methods.

We do not merely store character-specific visual and narrative facts but store RAG, to creative generation. This new use case provides consistency in generative work different to the regular question-answering work or summarization. In our ChromaDB our embedded profile creates are character profiles and we use sentence-transformers to embed the profile during prompt research, ensuring that the panels are coherent.

2.5 Prompt Engineering for Generative Models

Generation of high-quality images with minimal distortion by employing machine learning techniques to modify input images can be achieved by choosing the optimal style (filter) [2]. Prompt engineering is crucial in order to extract the best out of LLMs and diffusion models. The survey related by Sahoo et al. [8] categorized the prompt techniques and classified into zero-shot, few-shot and chain-of-thought. Chen et al. [2] demonstrated the significant influence positive prompts, descriptors of style, and time on the diffusion output. Vatsal and Dubey [10] specifically examined prompts of large language models and emphasized the significance of task-specific templates.

The Prompt optimisation Agent of comiccrafter does all this in: it injects comic specific style tags (comic book panel, graphic novel art), extracts cues of consistency using the RAG system, forms negative prompts to prevent common artifacts (distorted faces, inconsistent clothing). These measures are necessary to achieve quality outputs of free-tier image APIs unlikely to compete with the premium services.

2.6 Zero-Cost and Resource-Constrained AI Systems

Many generative AI studies presuppose the use of a big compute, yet practical application can run into a small budget. Our project aligns with the new trend of using the AI tool in various applications employing free-tier only. We use our tricks of optimization, smart caching and fallback on multiple providers and single shot generation to maintain quality with API quotas. To our understanding, ComicCrafter is the first known recorded system that demonstrates that you can create master-quality pages of comics by using technology, which is publicly available, with a careful orchestration of the said technologies.

3 Methodology

3.1 System Architecture Overview

It describes the overall structure of the system in question and highlights the connections among its components. There are four key components of ComicCrafter that include (1) the multi-agent orchestration layer, (2) a RAG-driven character

consistency module, (3) an image generation back-end and (4) an engine based on layout and assembly. The full pipeline is presented in Figure ??.

It works in the following way: first the text provided by the user enters the *Story Structuring Agent*, which splits the story into sceneries. *RAG system* pulls out and caches character data in a vector database. Then the *Agent of Prompt Optimization* synthesizes image prompts with the details of the retrieved characters in them. These prompts are sent to *Image Generation Backend* which is the one that draws the panels. Lastly, the *Layout Composition Agent* organizes the panels, includes speech bubbles and sews up the final pages of the comic.

3.2 LangGraph-Based Multi-agent Organization

Simplistic sets up in multi-agent systems are deployed with simple sequences or reactive patterns and these lack proper error tolerance and state monitoring. With LangChain, on top of which is LangGraph, we construct a directed acyclic graph (DAG) of agent steps conditionally borne plus automatic retries.

State Schema The workflow operates on a `ComicState` typed dictionary, defined as:

```
class ComicState(TypedDict):
    # Input
    story_text: str
    style: str
    max_scenes: Optional[int]
    add_dialogue: Optional[bool]

    # Intermediate results
    scenes: List[Scene]
    characters: List[Character]
    prompts: List[ImagePrompt]
    images: List[bytes]
    layouts: List[PageLayout]

    # Metadata and error tracking
    current_step: str
    retry_count: int
    errors: List[str]
    failed_images: List[int]

    # Final output
    output_path: Optional[Path]
    success: bool
```

This stateful representation enables nodes to access and modify shared context, facilitating seamless information flow across agent boundaries.

Algorithm 1 ComicCrafter LangGraph Workflow

```

1: Node 1: decompose_story → Parse text into structured scenes
2: Node 2: extract_characters → Identify characters and attributes
3: Node 3: generate_prompts → Create RAG-augmented image prompts
4: Node 4: generate_images → Produce panel images via diffusion API
5: Conditional Routing:
6: if failed_images.count < total_images × 0.5 and retry_count < 2 then
7:   → Node 5: retry_failed_images (simplified prompts)
8: else if failed_images.count = total_images then
9:   → Node 8: handle_error
10: else
11:   → Node 6: plan_layout (continue with partial success)
12: end if
13: Node 6: plan_layout → Determine panel arrangements
14: Node 7: assemble_comic → Composite final pages
15: Node 8: handle_error → Graceful failure recovery
16: return Final ComicState

```

Construction of a Workflow Graph LangGraph workflow consists of eight nodes which are interrelated with conditional edges.

This has a number of advantages over the linear methods:

- **Automatic retry feature:** when an image generating operation fails, the system continues to give more basic prompts and then resorts to place holder images.
- **Partial success processing:** when some of the images fail, the process proceeds, but only placeholders are used in empty panel positions.
- **State tracking:** The current state is updated in each node, which makes state debugging and performance analysis possible.

3.3 Specialized Agents

Story Structuring Agent Scene objects are the units of raw narrative text that the Story Structuring Agent divides raw narrative text into. Each scene holds:

- **description:** High-level scene summary
- **characters:** List of characters present
- **dialogue:** Spoken lines (if any)
- **action:** Physical actions occurring
- **setting:** Location and environment details

We employ Groq’s Llama 3.3 70B (or Google’s Gemini 2.0 Flash) with a carefully designed prompt template:

```

You are a comic script analyzer. Decompose the following
story into exactly {max_scenes} distinct scenes. For each

```

```
scene, provide:
1. Description (2-3 sentences)
2. Characters present (names only)
3. Dialogue (if applicable)
4. Primary action
5. Setting
```

```
Output as JSON array of scenes.
Story: {story_text}
```

The LLM response is parsed into structured **Scene** objects using Pydantic validation, ensuring schema compliance.

Character Extraction and RAG Integration Following scene decomposition, the system extracts **Character** entities:

- **name:** Character identifier
- **appearance:** Physical description (height, build, clothing, distinctive features)
- **personality:** Behavioral traits
- **role:** Narrative function (protagonist, antagonist, supporting)

Character descriptions are embedded using **sentence-transformers/all-MiniLM-L6-v2** and stored in ChromaDB. The embedding function is:

$$\mathbf{e}_c = \text{SentenceTransformer}(\text{concat}(\text{name}, \text{appearance}, \text{role}))$$

where $\mathbf{e}_c \in \mathbb{R}^{384}$ represents the character embedding. This vector store enables efficient similarity search during prompt generation.

Prompt Optimization Agent Each scene is already prepared to become a trigger in this agent. For every scene it:

1. **Retrieves character context:** by asking ChromaDB with character names so as to retrieve descriptions of appearances.
2. **Constructs a prompt of building base:** using the description of the scene, actions and setting.
3. **Includes style descriptions:** e.g. such as comic book style, graphic novel art, or professional illustration.
4. **Boom clicks the character details:** pulled out directly into the prompt.
5. **Mutually exclusive negatives prompts:** in order to prevent such common artifacts as distorted faces, additional limbs, blur, or poor quality.

The final prompt structure is:

```
{scene.action} in {scene.setting}. Characters:
{character_descriptions}. Style: {style}.
Negative: {negative_prompt}
```

This RAG-augmented prompting strategy significantly improves character consistency by explicitly providing visual attributes at each generation step.

Layout Composition Agent TheLayoutCompositionAgent determines panel arrangements based on scene count and narrative pacing. It supports multiple layout types:

- **SINGLE**: Full-page panel (climactic moments)
- **VERTICAL_2**: Two vertically stacked panels (sequential actions)
- **HORIZONTAL_2**: Two side-by-side panels (simultaneous events)
- **GRID_2X2**: Four-panel grid (rapid scene changes)
- **GRID_2X3**: Six-panel grid (standard comic page)

Layout selection employs rule-based heuristics:

- High-action scenes (keywords: "fight," "explosion," "chase") receive larger panels.
- Dialogue-heavy scenes are placed in smaller panels to accommodate speech bubbles.
- Page breaks occur after 4-6 panels to maintain readability.

3.4 Image Generation Backend

ComicCrafter uses three image generation backends and they are chosen using the `IMAGE_BACKEND` environment variable:

GGML (Local) Local flattening Use local inference with GGML using `stable-diffusion.cpp` quantized models (e.g. Stable Diffusion v1.5 in Q4.0 format). Advantages include zero API rate limits and full privacy. However, GGML backends have longer latency (10-15 seconds per image on CPU) and less support for such complex processes as ControlNet.

Implementation note: we first tried image to image refinement with the help of GGML to enhance character consistency through conditioning on previous panels. But this method was impossible because of the prohibitive latency (30-45 seconds per refinement cycles each), and thus impractical when generating multi-panels.

HuggingFace Inference API Builds on HuggingFace free-tier Inference API using such models as `stabilityai/stable-diffusion-xl-base-1.0`. Rate limits (about 100 requests/day of free accounts) require smart prompt optimization to minimize API calls. We adopt a back caching strategy: ground and setting prompts are hashed, as well as images that have already been generated with matching hashes are reused.

Replicate API Indicates access to higher level models (e.g. FLUX.1-dev) with superior prompt adherence. This however is limited by free-tier quotas, making this backend which is suitable to final refinement, as opposed to bulk generation.

3.5 Dialogue Integration and Text Overlay

There are two ways through which comic dialogues can be incorporated:

1. **In Image Generation:** Dialogue text is entrenched in image generation elicits (e.g., "character greeting with a Welcome, Hello"). This approach, while conceptually pleasing, faced huge constraints through the poor text generating abilities of existing diffusion models. Generated text often has spelling mistakes, imprecise fonts and mis-positioning.
2. **Post-generation Overlay:** Dialogue is a post-generation method in which dialogue is inserted after generating images using a **TextOverlay** engine made based on Pillow. Bubbles containing speech are placed aesthetically determined by panel size, and pasted in through readable fonts that are plain (Arial, Comic Sans MS). That style, though giving up beauty integration, has a guaranteed legibility and accuracy.

Limitations: In spite of having tried some specialized models and timely engineering, we discovered that the existing free-tier diffusion engines do not have the capacity to enable them to offer enough capacity ability to create speech bubbles of high quality in pictures themselves. Consequently, ComicCrafter goes to post-generation text overlay which is stable visual cohesion but not visual results.

3.6 FastAPI Web Interface

To enhance accessibility, ComicCrafter includes a FastAPI-based web interface with REST endpoints:

- POST `/api/generate`: Initiates comic generation (returns `job_id`)
- GET `/api/status/{job_id}`: Polls generation status
- GET `/api/output/{filename}`: Downloads completed pages
- POST `/api/workflow`: Executes LangGraph workflow with detailed step tracking

The web UI provides a form-based interface for story input, style customization, and real-time progress monitoring. Background task processing ensures non-blocking operation, allowing multiple concurrent generation jobs.

4 Experimental Setup and Results

4.1 Experimental Setup

Datasets The model is tested on three test sets:

1. **Too-Personal:** 15 user narratives (100-300 words a piece) genres served n genius: fantasy, science fiction, superhero, real-life.
2. **FlintstonesSV:** 20 episode descriptions of the story visualization Flintstones dataset [12].
3. **PororoSV:** 15 episodes description of Pororo dataset [6].

Evaluation Metrics Character Consistency:

- **DINO Similarity:** We retrieve character images features using DINO-ViT compute similarity between panels in DINO-ViT using cosine the same character. The scores above 5 consistency.
- **LPIPS Distance:** Learned Perceptual Image Patch Similarly perceptual distance. The decrease in LPIPS means that there is a more regular character rendering.

Narrative Coherence:

- **CLIP-T Score:** used to determine text-image correspondence by computing cosine similarity description of scenes CLIP embeddings vs generated image CLIP embeddings vs generated image Embedding of scene descriptions vs generated images.

Visual Quality:

- **Aesthetic Score:** Visual perception Uses a pretrained visual aesthetic predictor to score image quality (scale 1-10).
- **FID Score:** Fréchet Inception Distance between generated images and a reference comic dataset (better the lower).

System Performance:

- **Generation Success Rate:** Percentage of scenes generated without errors.
- **Latency:** Turbo round trip of full generation of comic.
- **API Efficiency:** API calls made per comic page.

Competitors We compare ComicCrafter to:

1. **Sequential Agent Coordination:** Innate pipeline without LangGraph (no retry logic).
2. **StoryDiffusion [12]:** Visualize stories with state-of-the-art appearance (needed the use of GPU velocity; we take the published results).
3. **DreamStory [6]:** Visualization of open-domain stories (proprietary DALL-E; results from paper).

4.2 Quantitative Results

The detailed analysis of the results of all metrics is provided in Table 1.

Key Findings:

1. **Character Consistency:** ComicCrafter has similarity in DINO of 0.72, significantly better than sequential coordination (0.58) by a figure of 24. While StoryDiffusion (0.84) is much better because fine-tuned consistency mechanisms in its approach of competitive performance without the use of GPU training.

Table 1: Comparative Evaluation Results (Mean $\pm$ Std Dev)

Metric	ComicCrafter	Sequential	StoryDiff	DreamStory
<i>Character Consistency</i>				
DINO Similarity $\uparrow$	0.72 ± 0.08	0.58 ± 0.12	0.84 ± 0.05	0.79 ± 0.06
LPIPS Distance $\downarrow$	0.34 ± 0.09	0.48 ± 0.11	0.21 ± 0.04	0.27 ± 0.05
<i>Narrative Coherence</i>				
CLIP-T Score $\uparrow$	0.68 ± 0.05	0.64 ± 0.07	0.71 ± 0.04	0.73 ± 0.04
<i>Visual Quality</i>				
Aesthetic Score $\uparrow$	6.2 ± 0.8	5.8 ± 1.1	7.1 ± 0.6	6.9 ± 0.7
FID Score $\downarrow$	42.3 ± 5.2	51.7 ± 6.8	28.4 ± 3.1	31.2 ± 3.9
<i>System Performance</i>				
Success Rate (%) $\uparrow$	89.2 ± 4.1	72.5 ± 8.3	95.0 ± 2.0	93.0 ± 2.5
Latency (sec/page) $\downarrow$	28.5 ± 6.2	25.1 ± 5.8	N/A	N/A
API Calls (per page) $\downarrow$	8.3 ± 1.2	9.7 ± 1.8	N/A	N/A

Table 2: Ablation Study: Component Contributions

Configuration	DINO Similarity	Success Rate (%)
Full System	0.72 ± 0.08	89.2 ± 4.1
w/o RAG (no character retrieval)	0.54 ± 0.11	88.5 ± 4.3
w/o LangGraph (sequential)	0.58 ± 0.12	72.5 ± 8.3
w/o Retry Logic	0.71 ± 0.09	76.3 ± 6.7
w/o Style Descriptors	0.69 ± 0.10	88.0 ± 4.5

2. **Narrative Coherence:** CLIP-t scores (0.68) are textimage similar to proprietary baselines, alignment. This validates our RAG augmented prompt generation strategy.
3. **Generation Reliability:** LangGraph orchestration agrees at 89.2% success rate compare to 72.5% sequential approach improvement by 23 per cent. Retriggering is accomplished automatically on Intermittent API failures.
4. **Efficiency:** Compared to sequential, ComicCrafter saves 14 percent of API calls techniques based on smart caching and one-shoot generation.

4.3 Ablation Study

Table 2 shows the ablation results that investigate the role of the important components.

Analysis:

- **RAGImpact:** The elimination of character retrieval decreases DINO similarity of 0.72 to 0.54 (25% degradation), which verifies the claim that RAG-augmented prompts are crucial for consistency.

Table 3: LLM Provider Comparison for Story Decomposition

Provider	Avg Response Time (s)	Parse Success (%)	JSON Validity (%)
Groq (Llama 3.3 70B)	2.3 ± 0.5	96.2	98.5
Google (Gemini 2.0 Flash)	3.8 ± 0.9	94.1	97.2
HuggingFace (Llama 2 7B)	5.2 ± 1.3	78.3	85.1

Table 4: Image Generation Backend Comparison

Backend	Latency (s/img)	Rate Limit	Aesthetic Score	Cost
GGML (local SD 1.5)	12.3 ± 2.1	None	5.8 ± 0.9	\$0
HuggingFace (SDXL)	4.2 ± 1.8	100/day	6.5 ± 0.7	\$0
Replicate (FLUX.1)	6.5 ± 2.3	50/day	7.1 ± 0.6	\$0 (trial)

- **LangGraphValue:** Sequential coordination reduces the potential changes in success rate (18.8 percent) proof of the usefulness of stateful workflow and recover methods.
- **Retry Logic:** The disablement of retry decreases the instance of success by 14.5 points (89.2% to 76.3%), shows the importance of graceful handling of errors.

4.4 LLM Provider Comparison

Table 3 provides a comparison between various free-tier providers of LLM used in story decomposition.

Key Findings:

- A balance between fastest response time and the fastest speed is **Groq Llama 3.3 70B**. Meanwhile, the largest success rate of parsing (2.3s) was found with the highest success rate (96.2%), and the highest level of JSON generation (98.5%). This renders it our default right to LLM.
- **Google Gemini 2.0 Flash** is providing similar quality but is slightly cheaper higher latency (3.8s).
- **HuggingFace Llama 2 7B** (free tier) does not cope with complex prompt adherence, which made it less successful in parsing (78.3%), and with frequent JSON formatting errors.

4.5 Image Backend Comparison

Various image generation backends are assessed in Table 4.

Analysis:

- **GGML** eliminates rate limits completely but has problems with high CPU-based latency (12.3s/image) and worse aesthetic (5.8/10) because of quantized models.

- **HuggingFace SDXL** provides the best latency quality-price tradeoff on free-tier though batch generation is limited by quotas per day, users.
- **Replicate FLUX.1** has the best aesthetic scores (7.1/10) and limited free quotas, and therefore it is only applicable to end-refinement passes.

4.6 Human Evaluation

A 25-participant user study was done where each of the 10 generated comics underwent an evaluation on four dimensions (1-5 Likert scale):

1. **Recognizability Character:** Can you recognize the same character in different panels? (Mean: 3.8 ± 0.6)
2. **Story Comprehension:** Have the story been told clearly well? (Mean: 4.1 ± 0.5)
3. **Visual Appeal:** To what extent are the images appealing? (Mean: 3.5 ± 0.7)
4. **Comprehensive Quality:** Do you like this comic? (Mean: 3.6 ± 0.6)

The findings show that ComicCrafter creates comics of a reasonable quality to share with friends, and understanding of stories is rated the highest (4.1/5). Image quality, visual appeal (3.5/5) is a problem, which is a limited feature of free-tier diffusion models.

5 Discussion and Limitations

5.1 Discussion of Results

By performing the experimental assessment, we confirm that ComicCrafter is able to meet its main goal which is to create the coherent, character-consistent comics within zero-cost boundaries. On the one hand, the multi-agent orchestration of LangGraph is very advantageous to the sequential methods, both in terms of generation reliability (89.2% success rate) and characters consistency (DINO similarity 0.72). The character memory based on RAG is very successful in enhancing consistency, and it is 25 times more effective than non-RAG baselines at the cost of no fine-tuning.

Comparative evaluation of the providers of the LLM shows that Groq Llama 3.3 70B is the most suitable to be used in situations when it comes to the tasks of decomposing stories on the basis of speed, accuracy and free-tier accessibility. Equally, the SDXL backend of HuggingFace offers the optimal latency-quality trade-off in the generation of images when limited by a rate.

5.2 Limitations

Regardless of such accomplishments, there are several weaknesses that ComicCrafter displays:

Lateness Image-to-Image Refinement The first hypothesis we had was that we would be able to improve consistency in the appearance of character images over multiple panels by using the image-to-image features of GGMI to refine them. Nonetheless, empirical experimentation showed that it was prohibitively slow on consumer grade CPUs: even a single iteration of the refinement process took 30-45 seconds, which is too slow to use with multi-panel generation. With 6 panels, it would take 3-4.5 min/refinement pass of a comic, which would not align with the usability goals. As a result, we have dropped this strategy in favor of RAG-augmented text prompting which is reasonably consistent with low latency overhead.

Future Direction: Refinement-based consistency (though with GPU-accelerated GGML inference or free tiers of cloud-based image-to-image APIs should emerge) might become practically useful.

Dialogue Bubble Generation The existing diffusion frameworks, including those on free-tier APIs, do not have a thorough text rendering system. In-image dialogue generation (putting text right in the Prompts such as character saying ,”Hello World,”) in our experiments delivering the strongest results consisted in continually generating:

- **Spelling mistakes:** Models often spell incorrect words or leave out letters in written text that they are prompted to write.
- **Font inconsistencies:** Generated texts have inconsistent fonts, font sizes and font orientations.
- **incorrect positioning:** Bubbles of speech are frequently placed over the face of characters, or sometimes that of panel lines.

The latter problems are because diffusion models have little knowledge of typographical structure and spatial reasoning. This causes ComicCrafter to default to the use of post-generation text overlay with Pillow, which makes the text readable but creates less visual fusion (overlapping bubbles can cover some background image or can look like they are floating).

Future Directions: This limitation can be overcome in special purpose text-conditioned diffusion models (e.g. extension of Glyph-SDXL) or in multi-stage pipelines (first generate the background, later conditionally add text).

A consistent and accommodating character Although the RAG based approach of ComicCrafter (0.72) is similar to DINO (0.72), it is not as good as the latest and cutting-edge model of StoryDiffusion (0.84), which uses fine-tuned self-attention schemes. RAG augmented prompts can offer textual description of characters, which can be erroneously understood by diffusion models (e.g., tall warrior in silver armor can produce different armor designs at different panels).

Future Direction: This gap could be reduced by means of either reference image conditioning (where APIs provide this) or by employing more hybrid solutions between RAG and lightweight adapter modules.

Free-Tier Rate Limits Although ComicCrafter operates within free-tier constraints, strict rate limits (e.g., HuggingFace’s 100 requests/day) can throttle batch generation for users creating multiple comics. Our caching and fallback strategies mitigate this partially, but prolonged use may require waiting periods or rotating API keys.

Future Direction: Expanded GGML support (more quantized models, faster inference on modern CPUs) would eliminate rate-limit dependencies entirely.

5.3 Broader Impact and Ethical Considerations

By democratizing access to AI-powered comic generation, ComicCrafter empowers independent creators, educators, and students who lack resources for expensive software or APIs. However, this accessibility also raises ethical concerns:

- **Copyright and Fair Use:** Generated comics based on copyrighted characters (e.g., from existing franchises) may infringe intellectual property rights. Users must ensure their narratives and character descriptions respect copyright boundaries.
- **Misinformation and Deepfakes:** Automated visual storytelling could facilitate creation of misleading narratives or synthetic media. We recommend watermarking generated comics to indicate AI authorship.
- **Artistic Labor:** Widespread adoption of automated comic generation may impact professional comic artists. We view ComicCrafter as a tool for rapid prototyping and education rather than a replacement for human creativity.

6 Conclusion and Future Work

This paper presented **ComicCrafter**, a novel zero-cost multi-agent AI system for automatic comic generation from text. By integrating LangGraph-based stateful workflows, RAG-powered character consistency, and multiple free-tier LLM and image generation backends, we demonstrate that sophisticated multimodal generation systems can operate entirely without proprietary resources or expensive infrastructure. Experimental evaluation shows that ComicCrafter achieves competitive character consistency (DINO similarity 0.72), strong narrative coherence (CLIP-T 0.68), and high reliability (89.2% success rate), outperforming sequential coordination approaches by 23%.

Our contributions include:

1. A comprehensive multi-agent architecture leveraging LangGraph for robust orchestration with conditional routing and automatic retry mechanisms.
2. A RAG-based character memory system using ChromaDB that maintains cross-panel consistency without requiring model fine-tuning.
3. Extensive comparative analysis of free-tier LLM providers and image generation backends, providing actionable insights for resource-constrained generative AI applications.
4. Identification and analysis of key limitations (GGML latency for image-to-image refinement, dialogue bubble generation challenges) that inform future research directions.

6.1 Future Work

Several promising directions for future research emerge from this work:

Advanced Consistency Mechanisms: Integrating reference image conditioning APIs (when available in free tiers) or developing lightweight LoRA adapters trained on character exemplars could push consistency beyond RAG-based prompting.

Multi-Page Narrative Planning: Extending the Layout Composition Agent to optimize panel arrangements across multiple pages, considering narrative pacing, cliffhangers, and visual flow.

Interactive Editing Interface: Developing a web-based editor allowing users to regenerate specific panels, adjust dialogue, or modify character descriptions iteratively.

Multilingual Support: Adapting the system to generate comics in multiple languages by incorporating multilingual LLMs and localized dialogue rendering.

Fine-Grained Style Control: Enabling users to specify per-panel artistic styles (e.g., manga for action scenes, watercolor for flashbacks) to enhance visual diversity.

Evaluation on Benchmark Datasets: Conducting comprehensive evaluation on established story visualization benchmarks (e.g., VIST, PororoSV) to enable direct comparison with state-of-the-art methods.

We release ComicCrafter as an open-source project (<https://github.com/Absirkhan/ComicCrafter>) to enable reproducibility, community-driven improvements, and democratized access to AI-powered creative tools. Our work demonstrates that thoughtful system design and intelligent resource orchestration can make advanced generative AI accessible to all, regardless of computational resources or budget constraints.

Acknowledgments

We thank Dr. Akhtar Jamil for guidance throughout this project, and the developers of Groq, Google AI Studio, HuggingFace, ChromaDB, LangChain, and LangGraph for providing free-tier access to their services, making this research possible.

References

1. Avrahami, O., Fried, O., Shechtman, E., Lischinski, D.: The chosen one: Consistent characters in diffusion models. In: ACM SIGGRAPH Conference Proceedings (2024)
2. Chen, Y., Xu, H., Li, J.: Unleashing the potential of prompt engineering in large language models: A comprehensive review. arXiv preprint arXiv:2310.14735 (2024)
3. Gao, L., Zhang, Y., Liu, X.: Retrieval-augmented generation for knowledge-intensive NLP tasks: A survey. arXiv preprint arXiv:2312.10997 (2024)
4. Guo, Y., Zhang, X., Liu, T.: Large language model based multi-agents: A survey of progress and challenges. arXiv preprint arXiv:2402.01680 (2024)

5. Han, J., Wu, C., Zhao, Y.: LLM multi-agent systems: Challenges and open problems. arXiv preprint arXiv:2402.03578 (2024)
6. He, J., Xu, K., Zhang, Y.: DreamStory: Open-domain story visualization by LLM-guided multi-agent collaboration. arXiv preprint arXiv:2407.12899 (2024)
7. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.T., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems (NeurIPS). pp. 9459–9474 (2020)
8. Sahoo, S., Mishra, R., Das, A.: A systematic survey of prompt engineering in large language models: Techniques and applications. arXiv preprint arXiv:2402.07927 (2024)
9. Tran, H., Nguyen, T., Pham, K.: Multi-agent collaboration mechanisms in large language models: A comprehensive survey. arXiv preprint arXiv:2501.06322 (2025)
10. Vatsal, P., Dubey, A.: A survey of prompt engineering methods in large language models for different NLP tasks. arXiv preprint arXiv:2407.12994 (2024)
11. Wang, Q., Zhang, F., Liu, J.: CharacterFactory: Sampling consistent characters with GANs in multimodal language models. IEEE Transactions on Image Processing (TIP) 34(2), 1219–1232 (2025)
12. Zhou, Y., Tang, L., Li, Z.: StoryDiffusion: Consistent self-attention for long-range image generation. In: Advances in Neural Information Processing Systems (NeurIPS) 2024 Spotlight (2024)